

Advanced Natural Language Processing

Deep Learning: Encoder-only Transformers: BERT

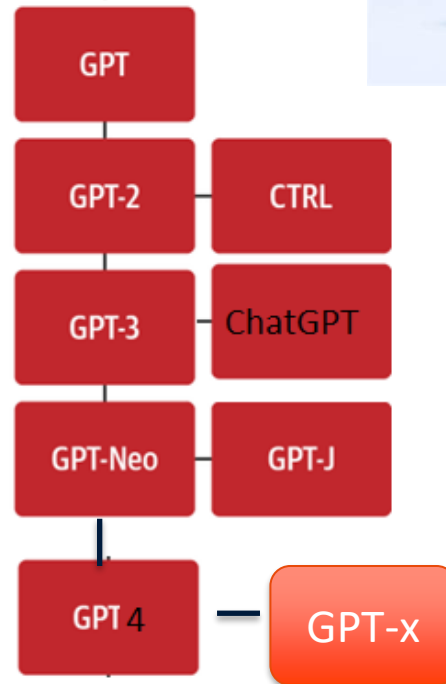
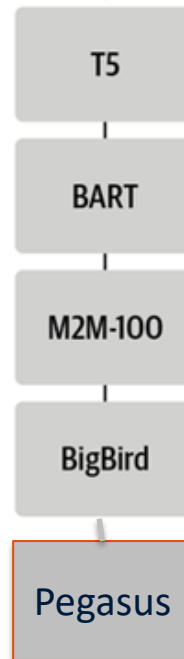
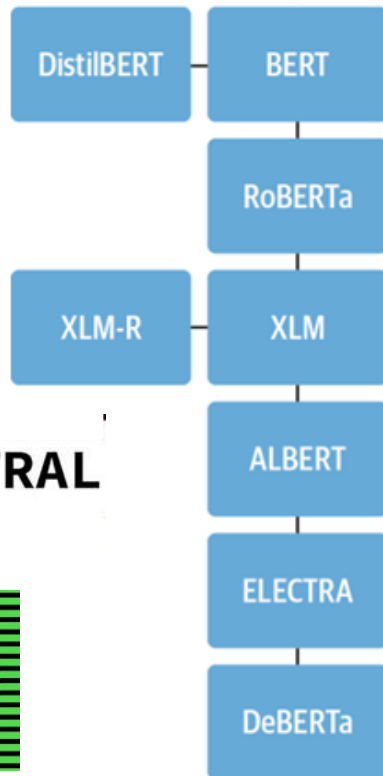
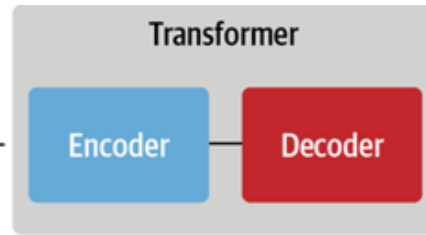
Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027



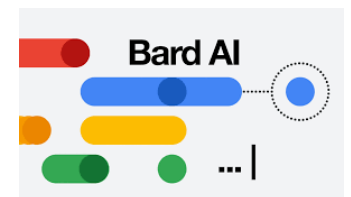
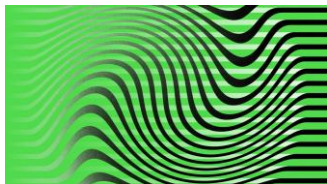
Maastricht University

The Transformer Zoo

Stanford
Alpaca



PyTorch



BERT

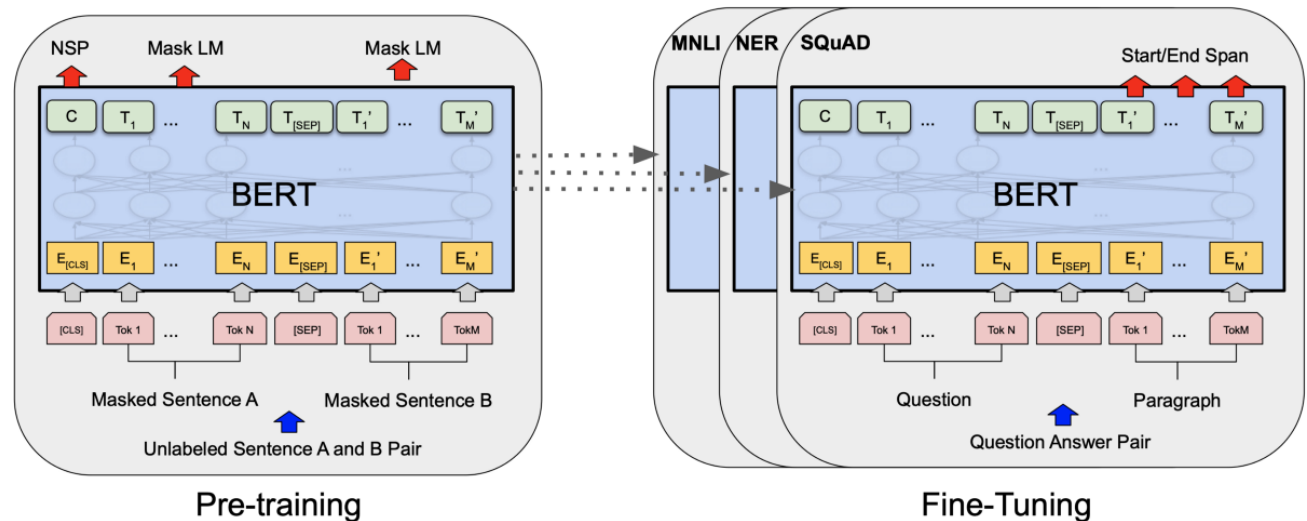
Bidirectional Encoder Representations
for Transformers



Maastricht University

Encoder only Model (such as BERT)

- Since BERT's goal is to generate a language model, only the encoder mechanism is necessary.
- BERT uses masked language models (LM), each masked token prediction is conditioned on the rest of the tokens in the sentence. These are received in the encoder, therefore you don't need a decoder.
- When we add a particular language representation task to the input (POS tag, Sentiment, NER role, ...), and predict that, we only use the encoder part.
- Encoder only best for CLASSIFICATION.



BERT

BERT:

- Uses the **encoder part** of the Transformer architecture, and a special form of language modeling called masked language modeling. The objective of masked language modeling is to predict randomly masked words in a text. For example, given a sentence like “I looked at my [MASK] and saw that [MASK] was late.” the model needs to predict the most likely candidates for the masked words that are denoted by [MASK]. BERT was pretrained on the BookCorpus and English Wikipedia.

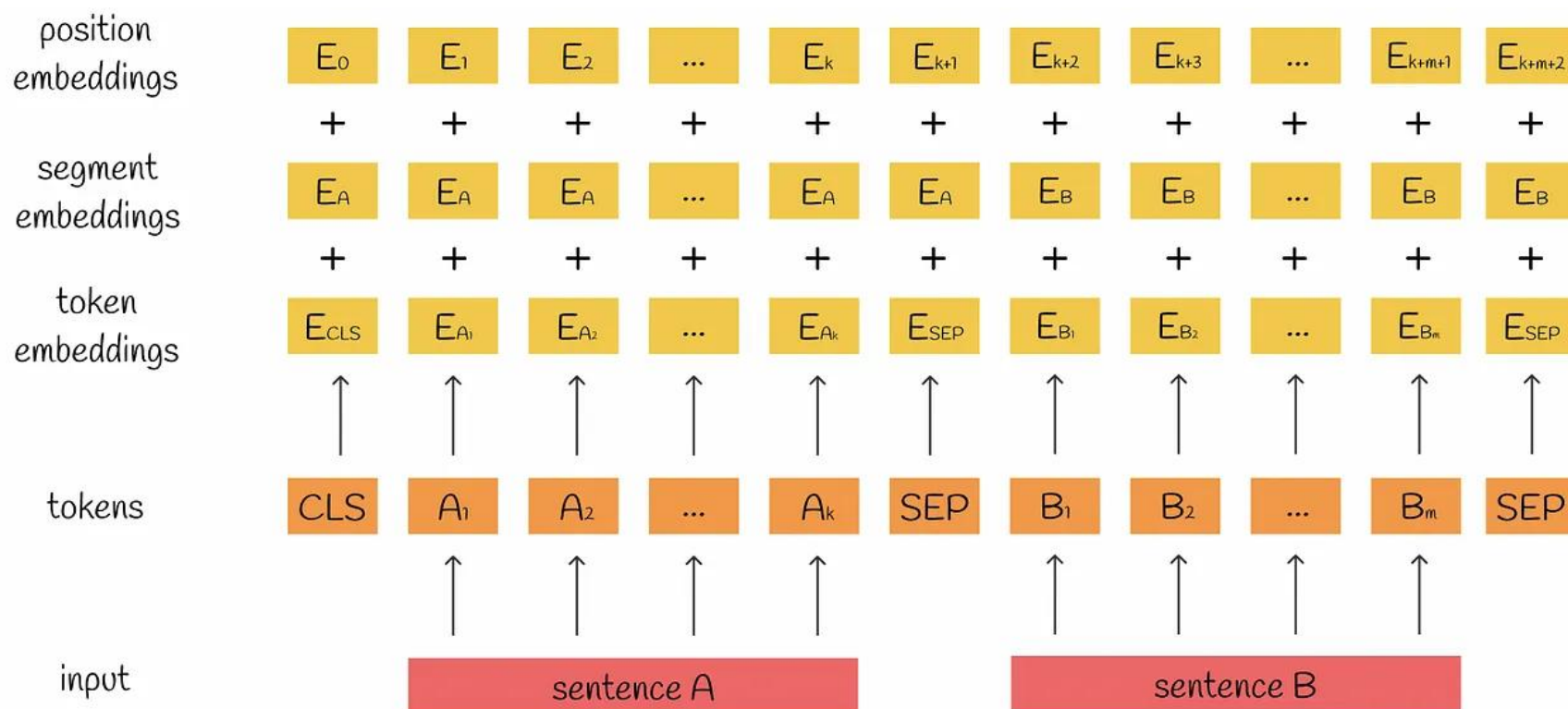
Source: <https://www.oreilly.com/library/view/natural-language-processing/9781098103231/ch01.html>

Pre-training and fine-tuning BERT: input

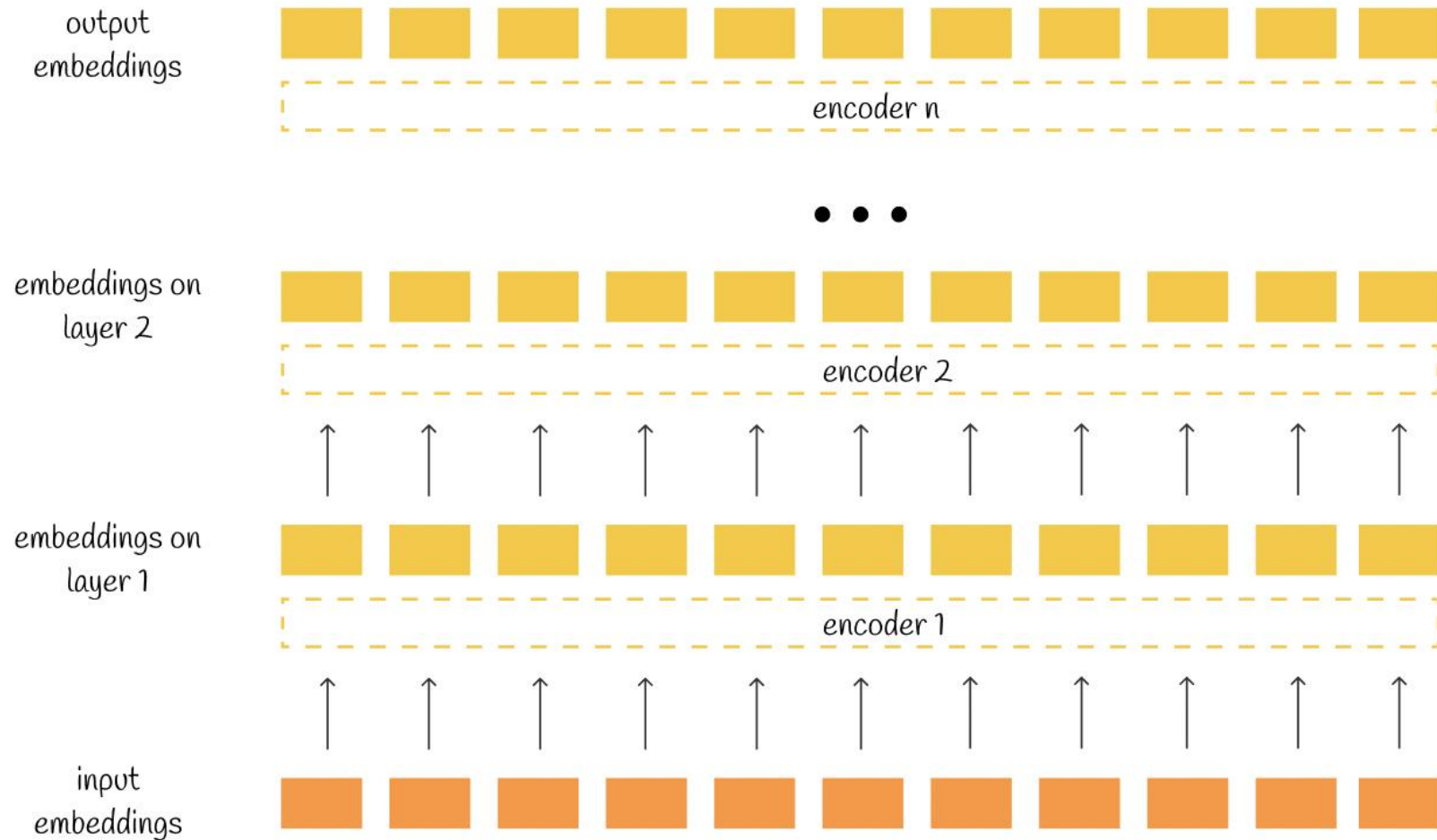
- For the input, BERT takes a single sequence or a pair of sequences. Each sequence is split into tokens.
- Additionally, two special tokens are passed to the input:
 - [CLS] — passed before the first sequence indicating its beginning: CLEAR SENTENCE. At the same time, [CLS] is also used for a classification objective during training. [SEP] — passed between sequences to indicate the end of the first sequence and the beginning of the second: SEPARATOR.
- Sending two sequences makes it possible for BERT to handle a variety of tasks where an input contains a pair of sequences (e.g. question and answer, hypothesis and premise, etc.).

Pre-training and fine-tuning BERT: Embeddings

After tokenization, an embedding is built for each token. To make input embeddings more representative, BERT constructs three types of embeddings for each token: **Token embeddings**, **segment embeddings** and **positional embeddings**.



Pre-training and Fine-Tuning BERT: Layers



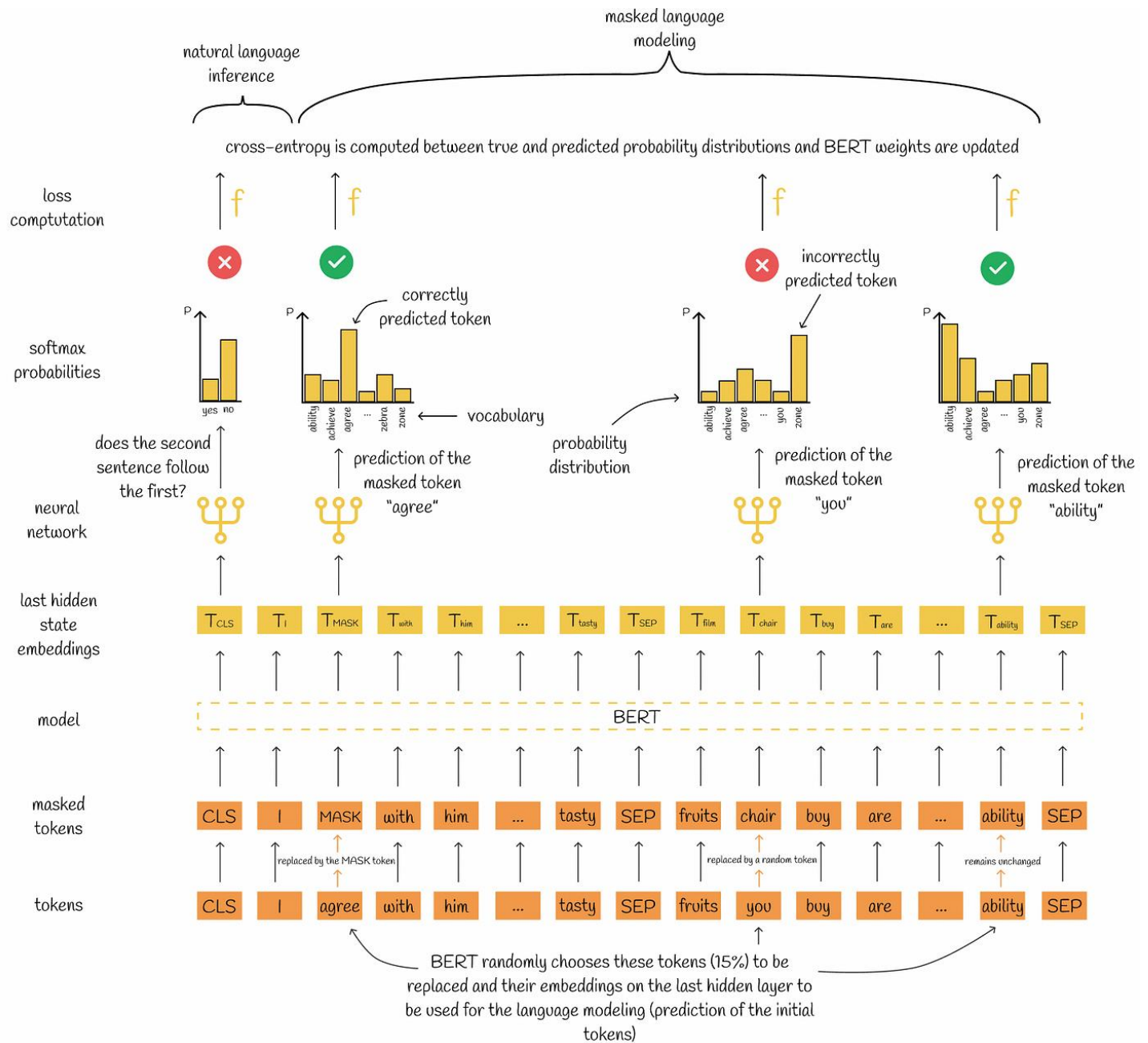
BERT: Natural Language Inference (NLI)

- BERT learns how to predict the second sequence follows the first. The prediction is made by using only the embedding from the final hidden state of the [CLS] token which is supposed to contain aggregated information from both sequences: **Question: How is this represented?**
- Similarly to Mask Language Modeling, a constructed probability distribution (binary in this case) is used to calculate the model's loss and update the weights of the model through backpropagation.
- For NLI, in the original BERT paper, it is recommended choosing 50% of pairs of sequences which follow each other in the corpus (positive pairs) and 50% of pairs where sequences are taken randomly from the corpus (negative pairs).



We will get back to this strategy later in the section on
“contrastive learning”

BERT: Pre-training for Natural Language Inference with Masked Language Modeling



Pre-Training and Fine-Tuning BERT

- **Pre-training:** BERT is pre-trained on unlabeled pair of sequences over two prediction tasks:
 - masked language modeling (MLM)
 - natural language inference (NLI).

We will discuss in detail what this is in the subsequent slides. For each pair of sequences, the model makes predictions for these two tasks and based on the loss values, it performs backpropagation to update weights.

- **Fine-tuning:** BERT is initialized with pre-trained weights during the above pre-training and can then be fine-tuned for a particular problem on labeled data during the fine-tuning process.



BERT: Masked-Language Modeling (MLM)

1. 15% of tokens are randomly masked. The masked tokens will be then predicted at the end of the iteration.

2. The masked tokens are replaced in one of three ways:

- -80% of the tokens are replaced by the **[MASK]** token.
 - Example: I bought a book → I bought a [MASK]
- -10% of the tokens are replaced by a random token.
 - Example: He is eating a fruit → He is drawing a fruit
- 10% of the tokens remain unchanged.
 - Example: A house is near me → A house is near me

3. All tokens are passed to the BERT model which outputs an embedding for each token it received as input.

4. Output embeddings corresponding to the tokens processed at step 2 are independently used to predict the masked tokens. The result of each prediction is a probability distribution across all the tokens in the vocabulary.

5. The cross-entropy loss is calculated by comparing probability distributions with the true masked tokens.

6. The model weights are updated by using backpropagation.

Masked Language Modeling

Masked language modeling is a fill-in-the-blank task, where a model uses the context words surrounding a [MASK] token to try to predict what the [MASK] word should be.

The model shown here is **BERT**, the first large transformer to be trained on this task. Enter text with one or more "[MASK]" tokens and the model will generate the most likely substitution for each.

Sentence:

It is [MASK] to
[MASK] that

Mask 1 Predictions:

11.5% **important**
11.1% **difficult**
8.8% **easy**
5.0% **possible**
3.5% **hard**

Mask 2 Predictions:

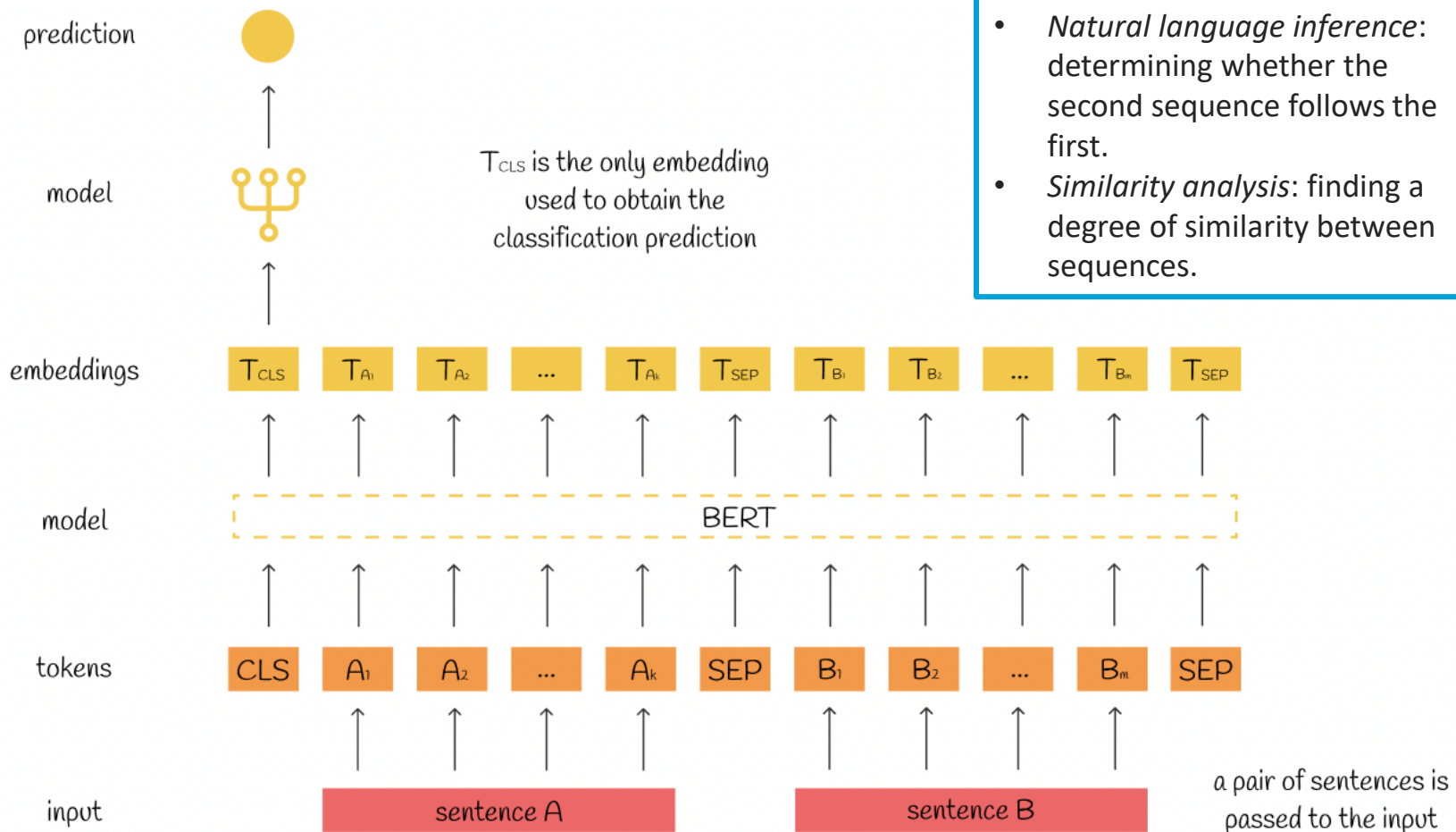
22.9% **say**
15.5% **note**
5.7% **see**
3.3% **suggest**
2.1% **conclude**

BERT (Bidirectional Encoder Representations from Transformers)

- BERT uses Transformer - an attention-based model with positional encodings to represent word positions. Builds on top of the bidirectional idea from ELMo, but uses the relatively new transformer architecture.
- BERT represents input as subwords and learns embeddings for subwords. So, it has a vocabulary that is about 30,000 for a model trained on a corpus with a large number of unique words (~ millions) - which is much smaller in contrast to a GloVe, Word2vec, or ELMo model trained on the same corpus.
- Due to the use of subwords, BERT can better deal with *Out of Vocabulary Words*.



BERT: Sentence Pair Classification

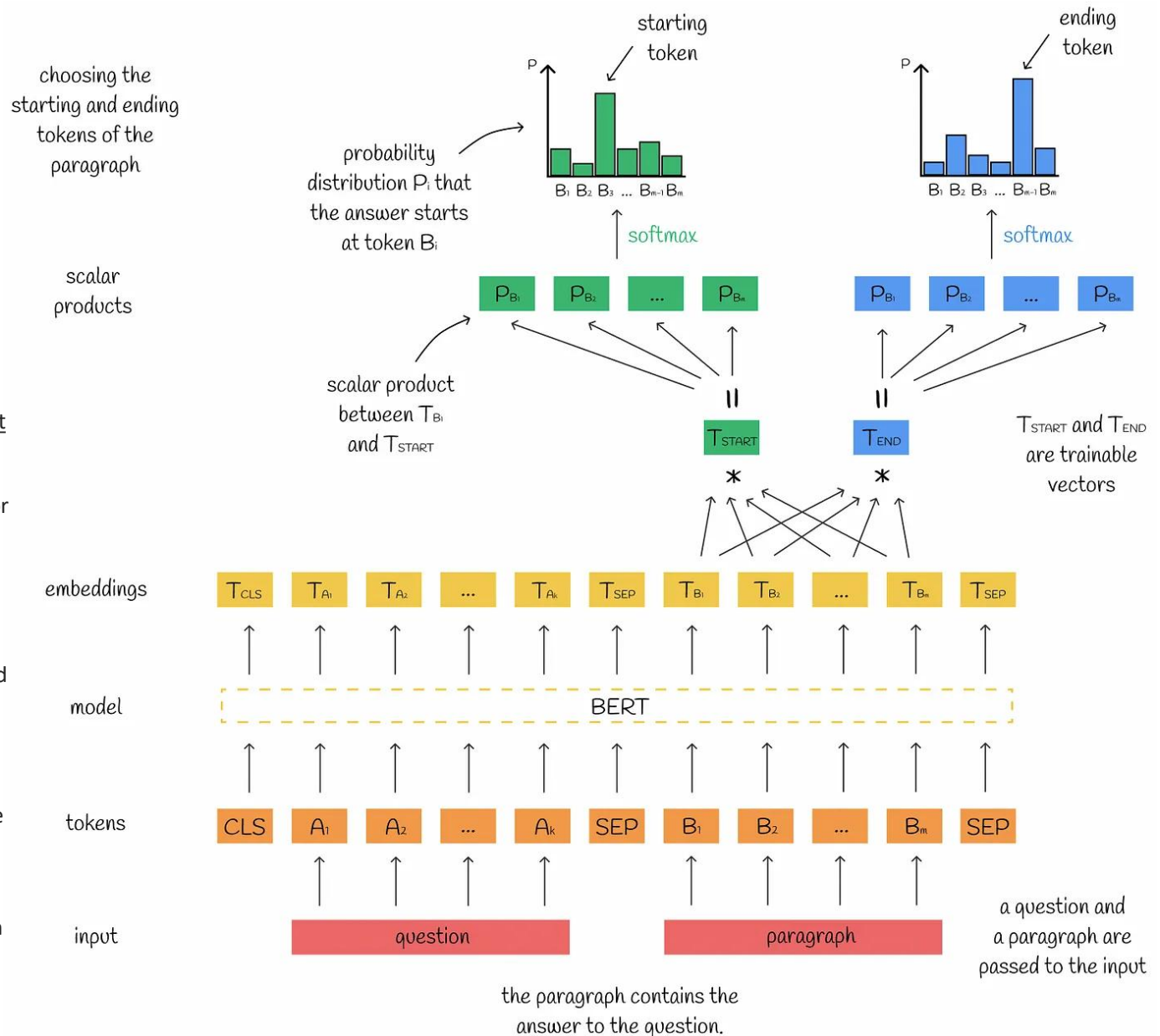


BERT:

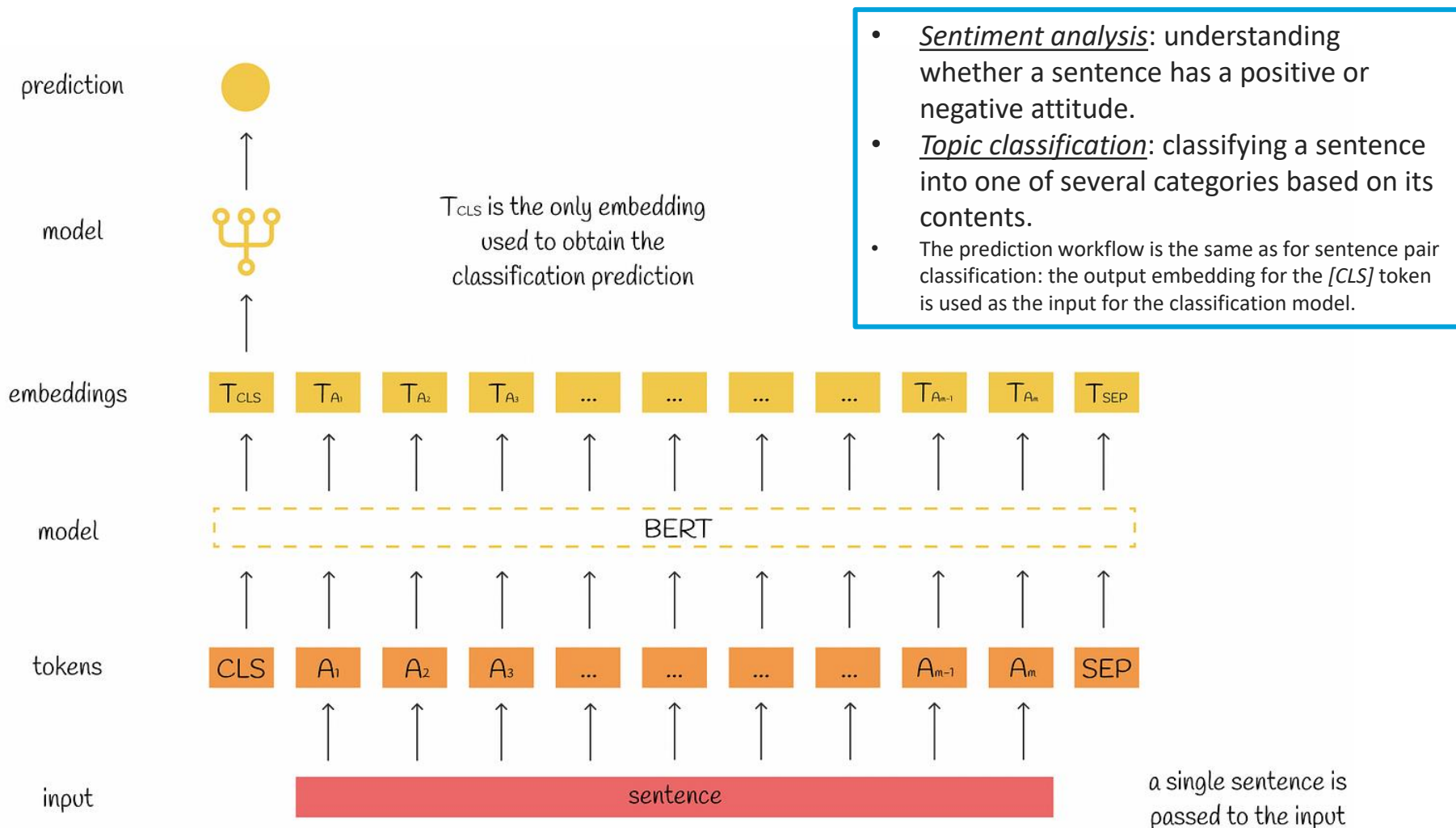
Question Answering Task

Find an answer in a text paragraph corresponding to a particular question.

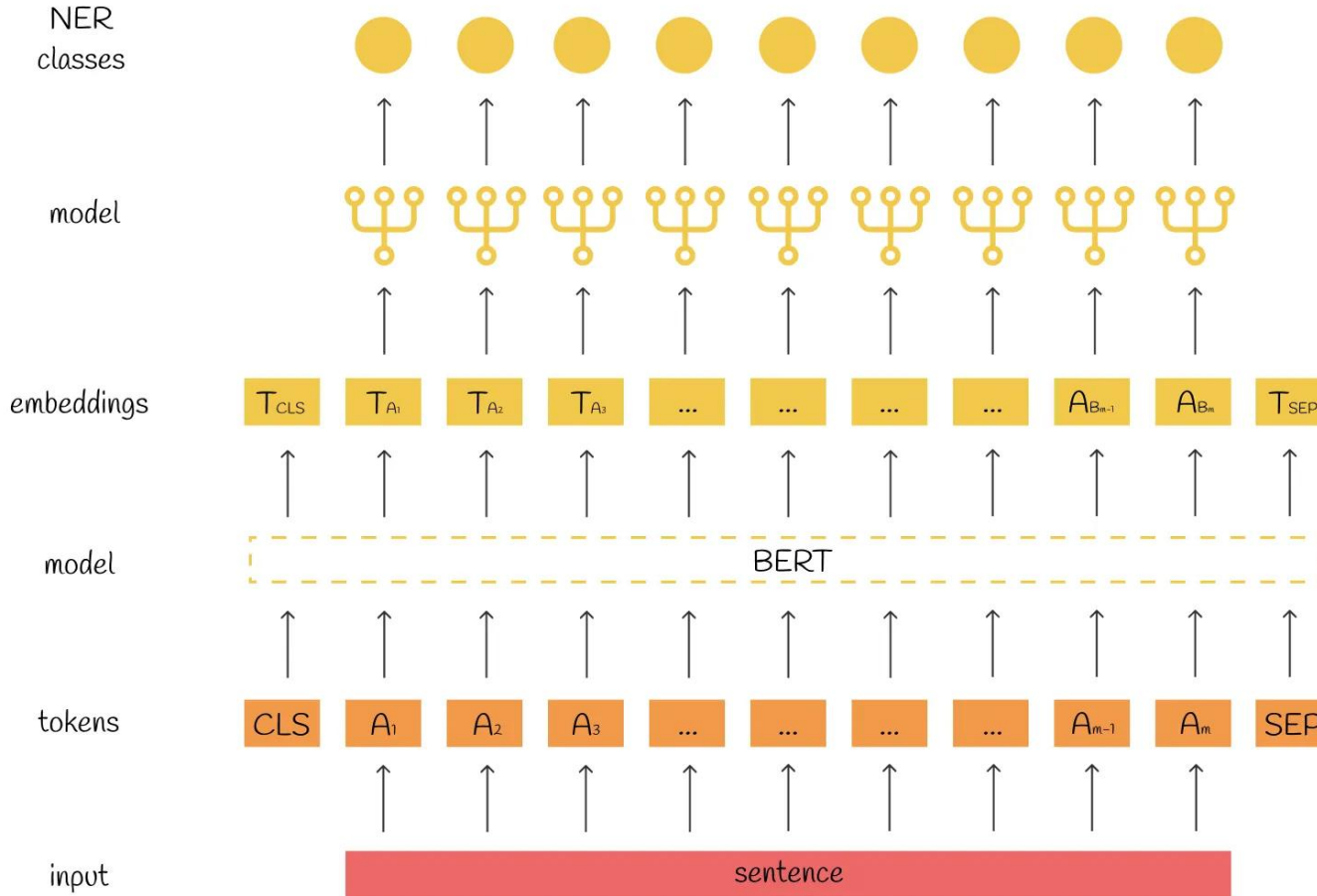
For finding a position of the start answer token in the paragraph, the scalar product between every output embedding and a special trainable vector T_{start} is calculated. For most cases when the model and the vector T_{start} are trained accordingly, the scalar product should be proportional to the likelihood that a corresponding token is in reality the start answer token. To normalize scalar products, they are then passed to the SoftMax function and can be thought as probabilities. The token embedding corresponding to the highest probability is predicted as the start answer token. Based on the true probability distribution, the loss value is calculated, and the backpropagation is performed. The analogous process is performed with the vector T_{end} for predicting the end token.



BERT: Single Sentence Classification



BERT: Single Sentence Tagging



Named entity recognition (NER) is a machine learning problem example, as is Part of Speech (POS) tagging.

Embeddings are computed for tokens of an input sentence, as usual. Then every embedding (except for $[CLS]$ and $[SEP]$) is passed independently to a model which maps each of them to a given NER class or POS tag (or not, if it cannot). Next, map every token of a sequence to one of respective entities or POS tags

a single sentence is passed to the input

Another Example of BERT Embedding

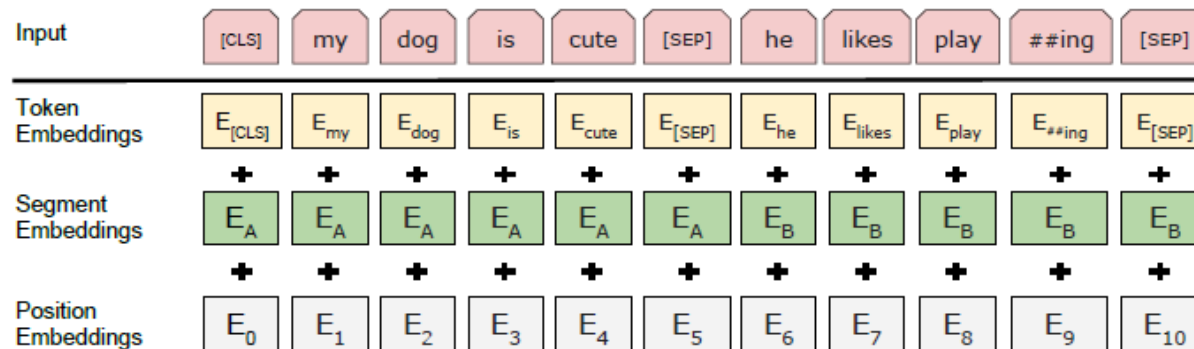


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

BERT (Sub-)words

Internally, BERT stores 3 types of (sub)words:

- 1) “Context-free” pre-trained embeddings
- 2) “Context-based” pre-trained embeddings
- 3) “Context-averaged” pre-training embeddings

Sub-words:

The original BERT vocabulary is fixed at 30.000 tokens. Words that are not part of the core vocabulary are represented as subwords and characters: notice how the word “embeddings” can be represented: ['em', '##bed', '##ding', '##s']

The WordPiece Tokenizer

- BERT uses what is called a WordPiece tokenizer.
 - Words are tokenized either into the full forms if it is a known word (part of a 30k vocabulary, then one word becomes one token), or
 - Words are tokenized into word pieces (if it is not part of the 30k vocabulary), then one word can be broken into multiple tokens.

Word	Token(s)
surf	['surf']
surfing	['surf', '##ing']
surfboarding	['surf', '##board', '##ing']
surfboard	['surf', '##board']
snowboard	['snow', '##board']
snowboarding	['snow', '##board', '##ing']
snow	['snow']
snowing	['snow', '##ing']

We will discuss this in more detail in the tutorial



Various BERT models

Model	Parameters (Millions)
BERT-Tiny	4.4
BERT-Mini	11.3
BERT-Small	29.1
BERT-Medium	41.7
BERT-Base	110.1
BERT-Large	340



Comparison	BERT October 11, 2018	RoBERTa July 26, 2019	DistilBERT October 2, 2019	ALBERT September 26, 2019
Parameters	Base: 110M Large: 340M	Base: 125 Large: 355	Base: 66	Base: 12M Large: 18M
Layers / Hidden Dimensions / Self-Attention Heads	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 6 / 768 / 12	Base: 12 / 768 / 12 Large: 24 / 1024 / 16
Training Time	Base: 8 x V100 x 12d Large: 280 x V100 x 1d	1024 x V100 x 1 day (4-5x more than BERT)	Base: 8 x V100 x 3.5d (4 times less than BERT)	[not given] Large: 1.7x faster
Performance	Outperforming SOTA in Oct 2018	88.5 on GLUE	97% of BERT-base's performance on GLUE	89.4 on GLUE
Pre-Training Data	BooksCorpus + English Wikipedia = 16 GB	BERT + CCNews + OpenWebText + Stories = 160 GB	BooksCorpus + English Wikipedia = 16 GB	BooksCorpus + English Wikipedia = 16 GB
Method	Bidirectional Transformer, MLM & NSP	BERT without NSP, Using Dynamic Masking	BERT Distillation	BERT with reduced parameters & SOP (not NSP)

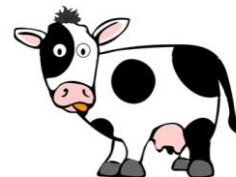
BERT and Human Languages

- BERT is language dependent. Original BERT models will only work well on English.
- m-BERT: Multilingual BERT (M-BERT), released by Devlin et al. (2018) as a single language model pre-trained from monolingual corpora in 104 languages. See <https://aclanthology.org/P19-1493/> for an in-depth evaluation.
- XLM-BERT: Cross-lingual BERT model. XLM-BERT uses a known pre-processing technique (BPE) and a dual-language training mechanism with BERT in order to learn relations between words in different languages. See: <https://arxiv.org/abs/1901.07291> for more information.
- Language Specific BERT: the highest fidelity and odds for success for a specific language: BERTje (NL), CamenBERT (FR), FlauBERT (fr), BanglaBERT, indoBERT, PhoBERT (Vietnamese), EstBERT (Estonian), AraBERT, KoBERT (Korean), BERTurk (Turkish),



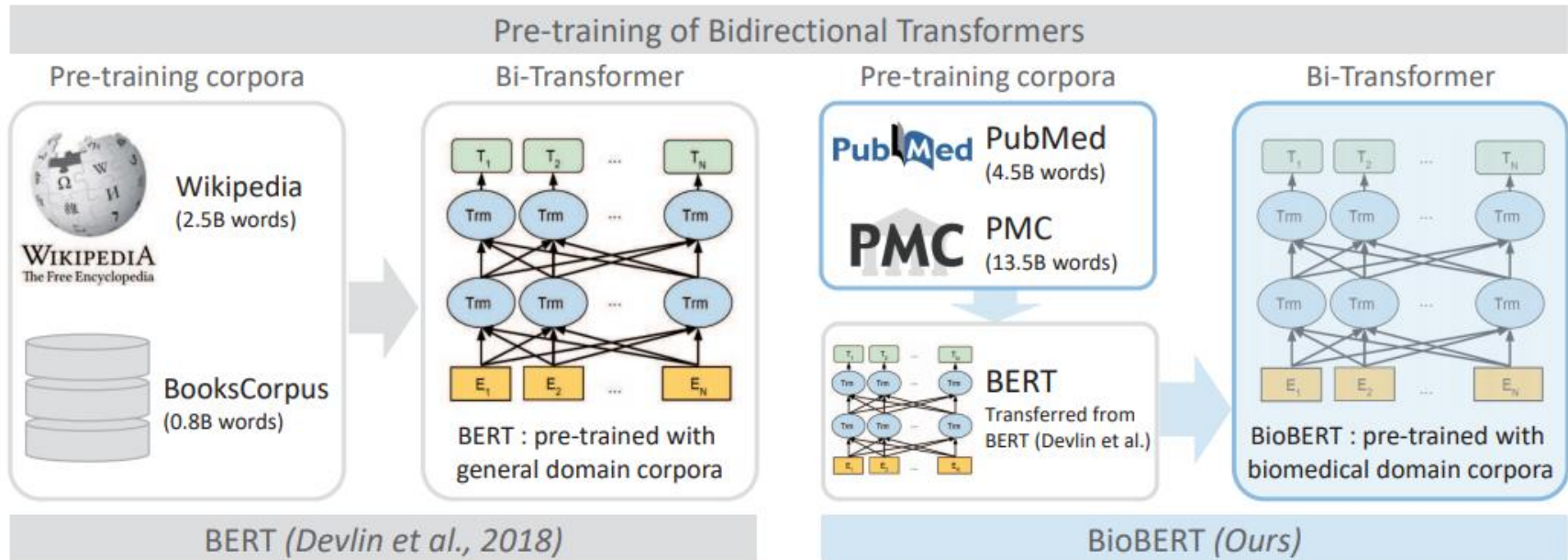
BERTje: A Dutch BERT model

BERTje is a Dutch pre-trained BERT model developed at the University of Groningen.



For details, check out our paper on arxiv: <https://arxiv.org/abs/1912.09582>

Domain & Language Adaptation with BERT



Evaluation	BERT	RoBERTa	DistilBERT	ALBERT	Bidirectional NN	SVM
AUC	0.8763	0.9011	0.8761	0.8757	0.8541	0.8561
F1-Score	0.425	0.4539	0.434	0.4717	0.37	0.47
MCC	0.4086	0.4528	0.4141	0.4456	0.29	0.41
Runtimes	18min	18min	16min	15min	20min	5min

Social Media T-PTLMs

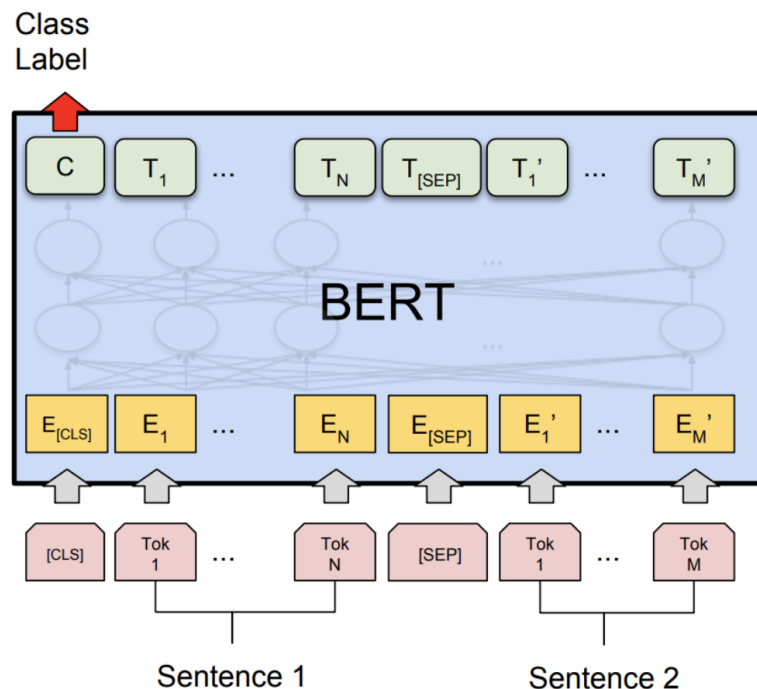
Model
HateBERT
RoBERTa-twitter
RoBERTa-reviews
BERT-SentiX
XLNet-R-twitter
BERTweet
BERTweet-COVID19
CT-BERT

Fine Tuning BERT

- Q: Why are there so many BERT models?
- A: Because it is easy to fine-tune the original BERT model.

We will look at this in more detail in the tutorials how to do this.

In the lecture “Fine Tuning Large-Language Models”, we will take a close look at training and fine tuning.



ModernBERT

**BERT (2018) →
ModernBERT
(2024)**

- As of 2026, encoder-only models remain the standard choice for classification, retrieval, and structured extraction (they're significantly faster per token than decoder LLMs), and ModernBERT is still cited as the reference modern encoder — it's routinely recommended as the default upgrade path from BERT for RAG and semantic-search pipelines rather than reaching for a generative model.

“Smarter, Better, Faster,
Longer: A Modern
Bidirectional Encoder for Fast,
Memory Efficient, and Long
Context Finetuning and
Inference”

Component	BERT	ModernBERT	Why it matters
Context length	512 tokens	8,192 tokens (16x)	Full documents, not just chunks
Position encoding	Learned absolute	RoPE (rotary)	Better long-context generalization
Activation	GeLU	GeGLU	Higher quality per parameter
Attention	Full attention every layer	Alternates global/local (every 3rd layer global)	Much faster, same accuracy
Training data	~3B tokens, no code	2T tokens, incl. code	Works on code search/retrieval tasks
Tokenizer	Original BERT vocab (2018)	Modern BPE (OLMo-based, 50k vocab)	Better efficiency, handles code syntax
Padding	Padded batches (wasted compute)	Unpadded + sequence packing	10–20% speed gain
Hardware design	Not GPU-optimized	Tuned for common GPU tile sizes	2x+ faster inference
Result	GLUE ~85, 512-token ceiling	GLUE 88+, SOTA retrieval & code, 2-3x faster at long context	Pareto improvement: faster <i>and</i> more accurate

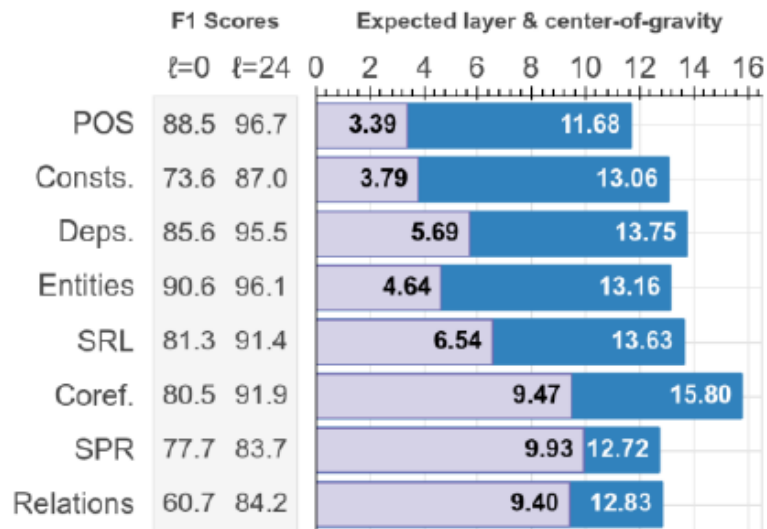
From Transformers back to Computational Linguistics

Where is the morphology, syntax, semantics, coreferences, anaphora, etc.?



Maastricht University

Different NLP Tasks in BERT Layers

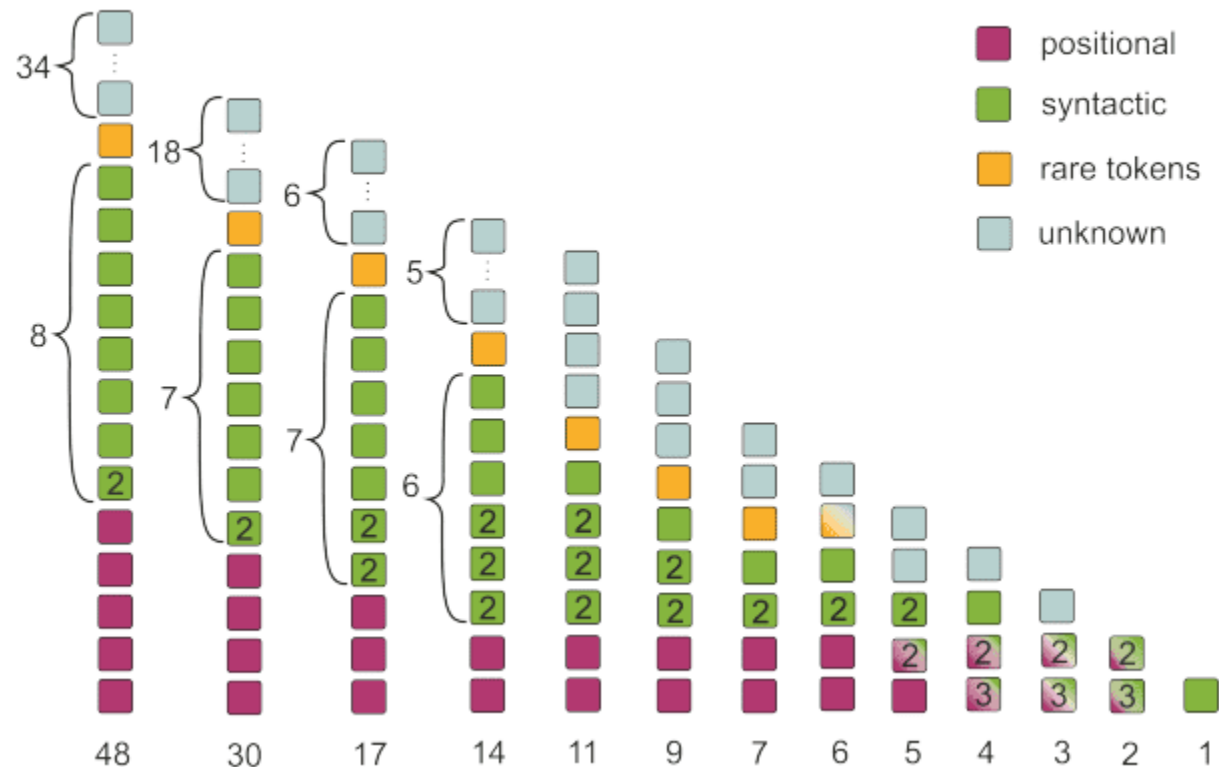


BERT layers vs NLP pipeline sub-tasks
[Tenney et al., 2019]

POS = Part of Speech Tagging (noun Verb, Determiner, ...)
 Consts = Constituency Parsing (NP, VP)
 Deps = Dependency Parsing
 Entities = Named Entity Extraction
 SRL = Semantic Role Labeling
 Coref = Coreference and Anaphora resolutions
 SPR = SPR data set for Semantic and Prototypes Relations
 Relations = Complex relations (RDF – Subject -Predicate- Object)

Specialized Heads in MH-SA

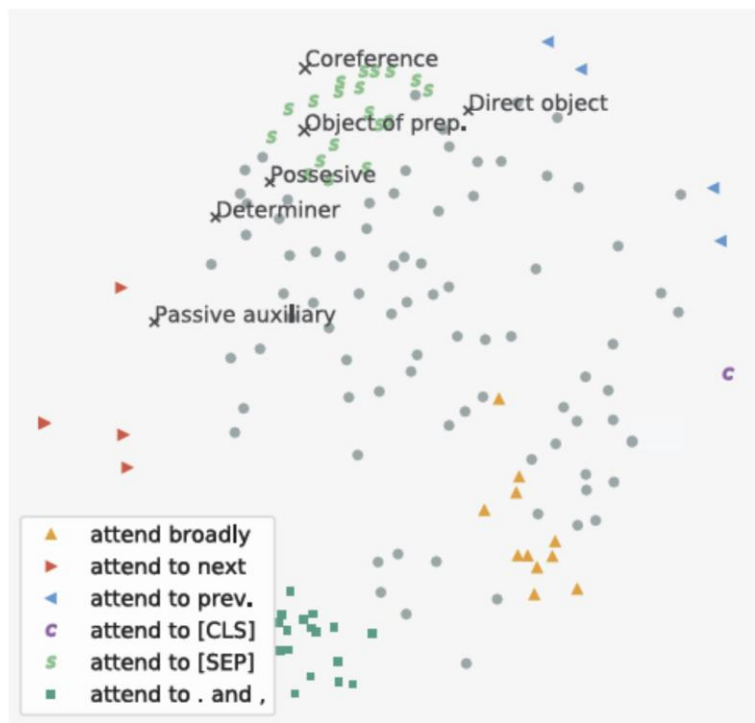
- Interestingly, when looking in more detail to the individual heads, we can observe that certain heads perform certain operations.
- The other heads can often be pruned (distillation) as they do not contribute.



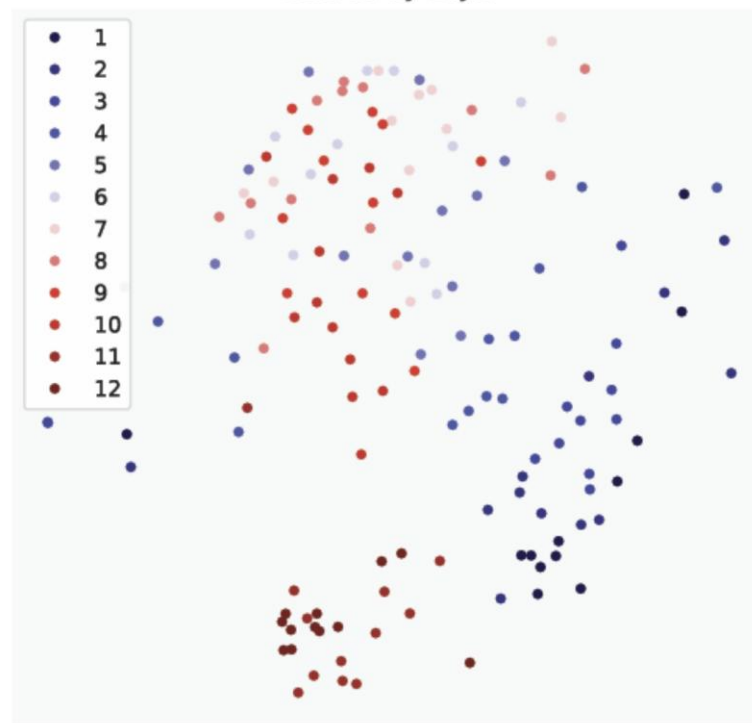
What does BERT look at? Attention explained

Embedded BERT attention heads

Behaviors



Colored by Layer

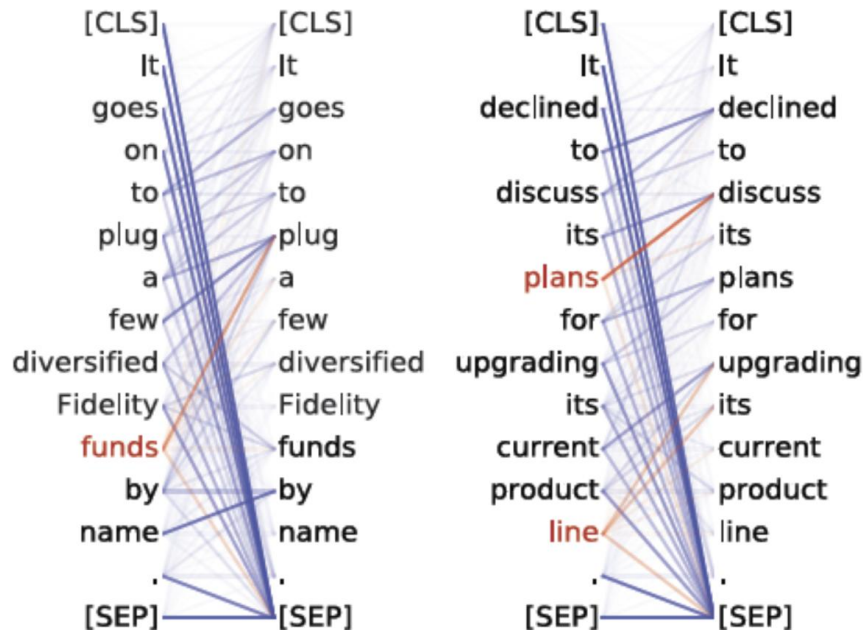


What does BERT look at? An Analysis of BERT's Attention. Clark et al. Proceedings of the Second BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP, pages 276–286. Florence, Italy, August 1, 2019. Association for Computational Linguistics

Direct Objects and Noun Modifiers

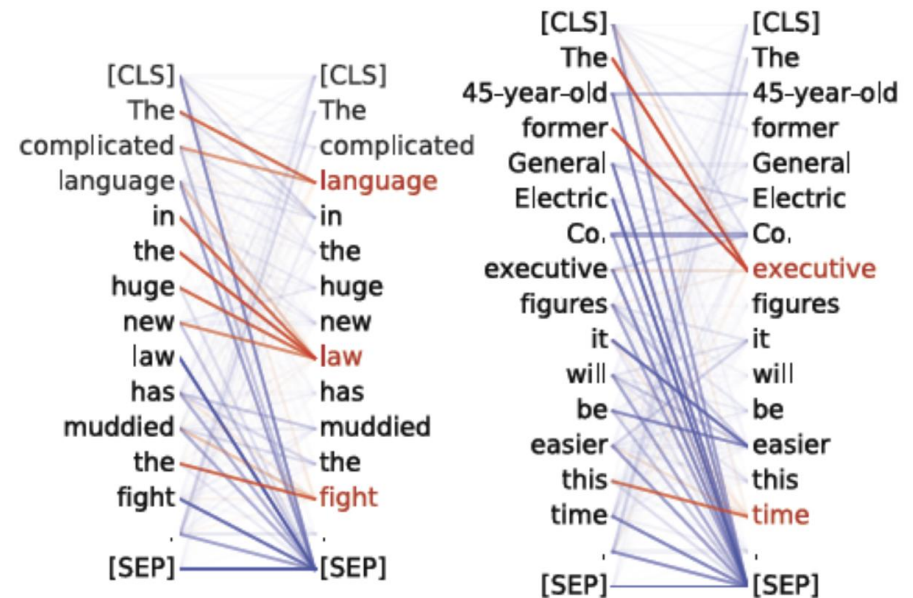
Head 8-10

- **Direct objects** attend to their verbs
- 86.8% accuracy at the dobj relation



Head 8-11

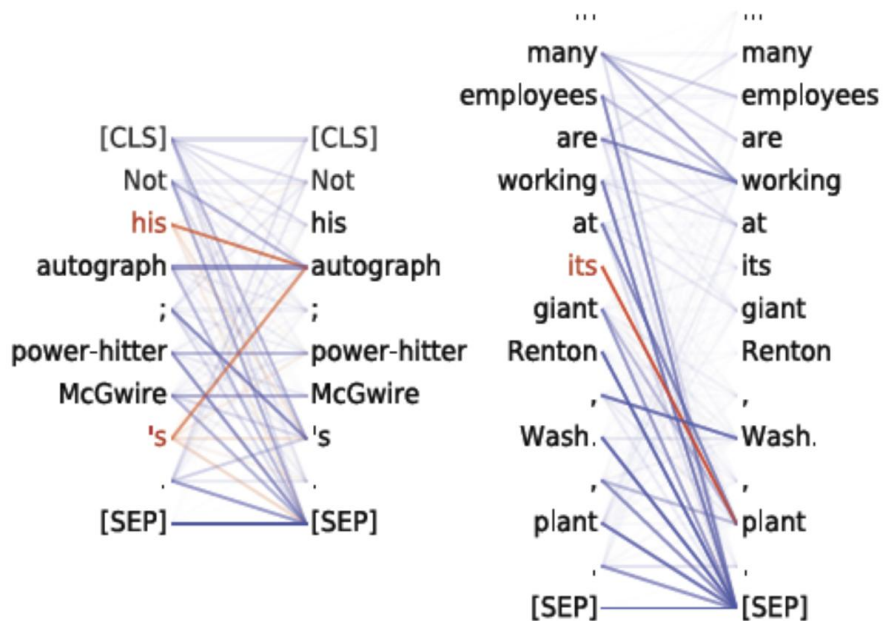
- **Noun modifiers** (e.g., determiners) attend to their noun
- 94.3% accuracy at the det relation



Possessive Pronouns and Passive Auxiliary Verbs

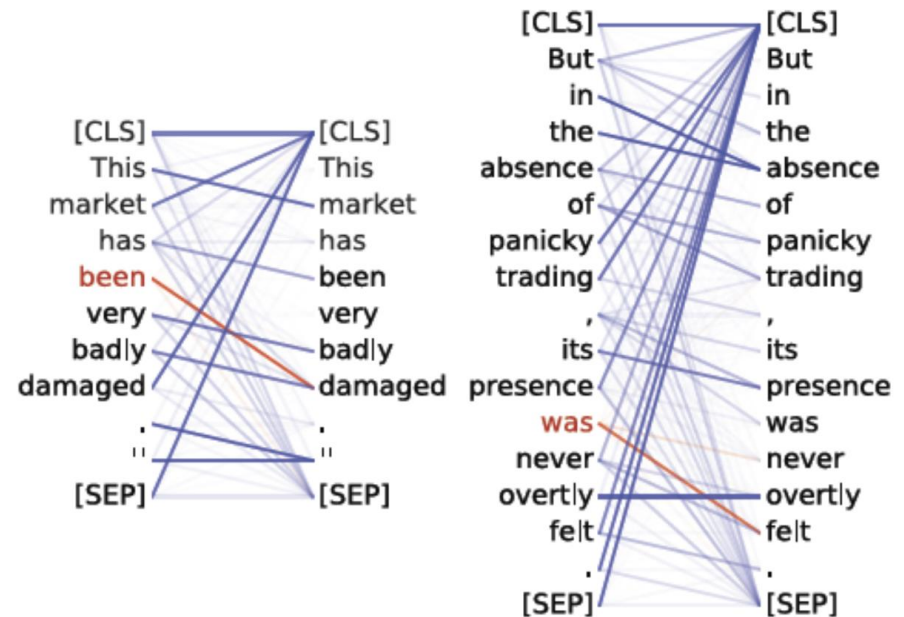
Head 7-6

- **Possessive pronouns** and apostrophes attend to the head of the corresponding NP
- 80.5% accuracy at the poss relation



Head 4-10

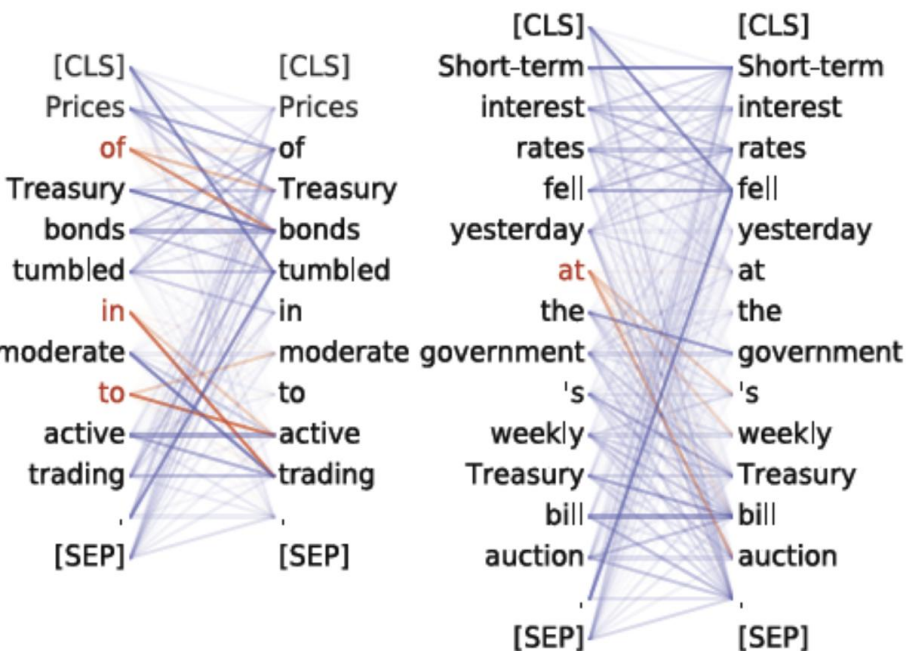
- **Passive auxiliary verbs** attend to the verb they modify
- 82.5% accuracy at the auxpass relation



Prepositions and Coreferents

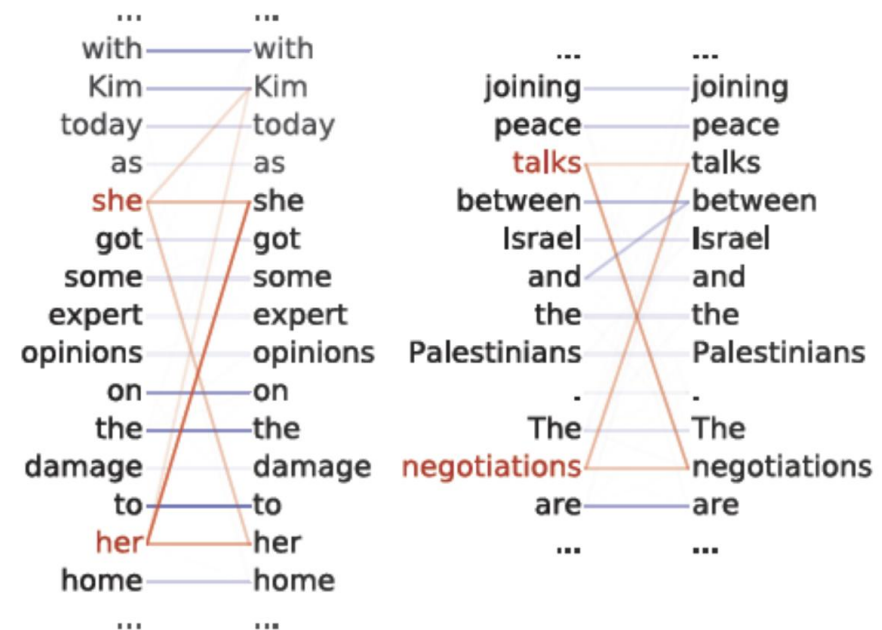
Head 9-6

- **Prepositions** attend to their objects
- 76.3% accuracy at the pobj relation



Head 5-4

- **Coreferent** mentions attend to their antecedents
- 65.1% accuracy at linking the head of a coreferent mention to the head of an antecedent



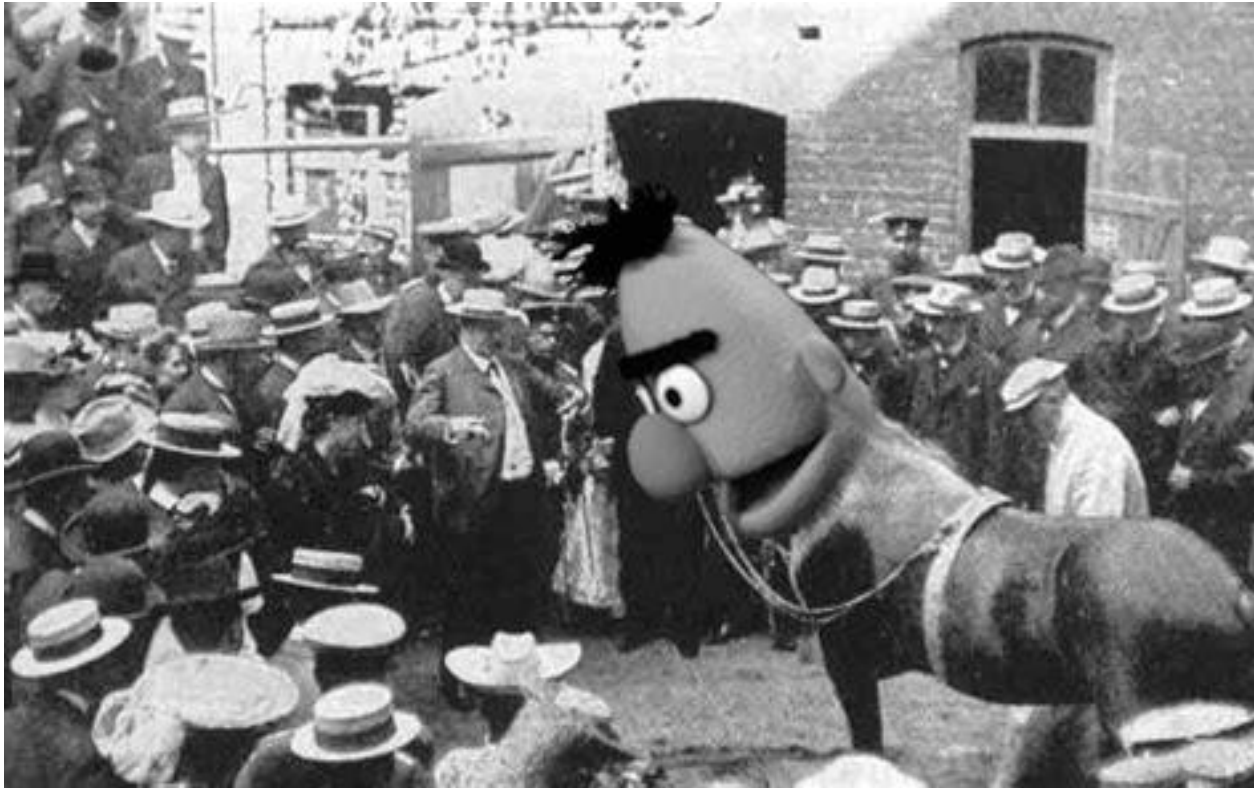
Be Aware: “Negation handling with Deep Learning does not always work ...”

Context	BERT _{LARGE} predictions
<i>A robin is a ____</i>	<i>bird, robin, person, hunter, pigeon</i>
<i>A daisy is a ____</i>	<i>daisy, rose, flower, berry, tree</i>
<i>A hammer is a ____</i>	<i>hammer, tool, weapon, nail, device</i>
<i>A hammer is an ____</i>	<i>object, instrument, axe, implement, explosive</i>
<i>A robin is not a ____</i>	<i>robin, bird, penguin, man, fly</i>
<i>A daisy is not a ____</i>	<i>daisy, rose, flower, lily, cherry</i>
<i>A hammer is not a ____</i>	<i>hammer, weapon, tool, gun, rock</i>
<i>A hammer is not an ____</i>	<i>object, instrument, axe, animal, artifact</i>

Table 13: NEG-88-SIMP predictions by BERT_{LARGE}

Source: What BERT is not: Lessons from a new suite of psycholinguistic diagnostics for language models.
Allyson Ettinger, July 2019. <https://arxiv.org/abs/1907.13528>

The Clever Hanns Effect?



Memorizing versus understanding?

Questions ...



Maastricht University

Exercise 1: BERT Input Representation and Pre-Training Objectives

BERT (Bidirectional Encoder Representations from Transformers) builds deep representations using an encoder-only stack trained on two self-supervised objectives: Masked Language Modeling (MLM) and Natural Language Inference / Next Sentence Prediction (NLI/NSP).

- For any given input sequence, BERT computes an input vector by summing three distinct embeddings element-wise: **Token Embeddings**, **Segment Embeddings**, and **Position Embeddings**. Explain the specific function of each of these three embeddings, and explain why the special tokens [CLS] and [SEP] are required in the token sequence.
- Under the Masked Language Modeling (MLM) objective, exactly 15% of tokens are selected for prediction. According to Devlin et al. (2018), what are the three replacement rules applied to these selected tokens (include their exact percentages), and why doesn't BERT simply replace all selected tokens with the [MASK] token?
- In the pre-training task for sentence pairs, explain how the final hidden state vector corresponding to the [CLS] token (T_{CLS}) is utilized to decide whether Sentence B follows Sentence A. What loss function is used to update the model weights for both MLM and NLI?

Exercise 2: Downstream Task Adaptation and Fine-Tuning Architectures

BERT can be adapted to various NLP classification and tagging tasks by attaching a task-specific classification head to its output embeddings.

- Contrast how BERT handles **Single Sentence Classification** (e.g., sentiment analysis) versus **Single Sentence Tagging** (e.g., Named Entity Recognition or POS tagging):
 - Which output vector is passed to the classification head for sentence classification?
 - How are token-level embeddings ($T_1, \dots, T_m$) mapped to classes in sequence tagging tasks, and which token embeddings are omitted from this prediction?
- For extractive **Question Answering** on the SQuAD benchmark, BERT is fed the question and the reference passage separated by [SEP]. Explain how the start position and end position of the answer span are determined within the paragraph tokens using the learned parameter vectors T_{START} and T_{END} .
- Review the model comparison table on slide 26. What structural improvements and training modifications distinguish **RoBERTa** and **DistilBERT** from original **BERT-Base** in terms of training data, parameter count, and pre-training objectives?

Exercise 3: BERT Layers and Specialized Attention Heads

Recent interpretability and probing work (Tenney et al., 2019; Clark et al., 2019) investigates how linguistic hierarchy emerges across BERT's internal representations.

- Examine the layer-probing chart from Tenney et al. At what layers do low-level syntactic tasks (such as POS tagging and constituency parsing) reach their center-of-gravity compared to higher-level semantic tasks (such as coreference resolution and relation extraction)? How does this relate to the classical 1970–1990 NLP pipeline?
- The lecture shows that individual self-attention heads specialize in specific grammatical relations. Identify two examples from the lecture where specific heads track particular syntactic relations (e.g., direct objects attending to verbs, possessive pronouns attending to noun phrases).
- Research by Voita et al. (2019) demonstrates that many attention heads can be pruned without degrading performance. If only a subset of specialized heads performs the primary grammatical work, why does BERT train with 12 to 16 heads per layer initially instead of training only the necessary heads from the start?

Discussion



On the [CLS] Token Bottleneck: Can a Single Vector Compress a Sentence?

In BERT's classification tasks—such as sentiment analysis or natural language inference—the model routes the entire sequence through deep bidirectional self-attention, but then throws away all token embeddings except for the final hidden state of the [CLS] token (T_{CLS}) to make its prediction.

Yet in an earlier lecture, we criticized sequence-to-sequence RNNs because squeezing an entire sequence into a single vector bottleneck caused catastrophic information loss on longer sentences. If compressing a sequence into a single vector was an architectural flaw in RNNs, why is relying exclusively on T_{CLS} considered a breakthrough in BERT?

Did bidirectional self-attention genuinely solve the information bottleneck, or did we just move the bottleneck from an encoder's hidden recurrent state to a special classification token?

On the Accidental Linguistic Pipeline: Objective Law or Statistical Mirage?

Tenney et al. (2019) demonstrated that BERT's 24 layers organically rediscover the classical 1970s NLP pipeline: morphology and POS tags solidify at layers 3–4, syntax in the middle layers, and coreference or relation extraction in the final layers. Furthermore, Clark et al. (2019) revealed that individual self-attention heads independently specialize into tracking direct objects, prepositions, or possessive pronouns.

Because BERT was trained strictly on an unannotated, self-supervised objective (filling in [MASK] tokens and predicting sentence pairs) with zero explicit grammar rules, does this prove that Chomskyan syntactic hierarchy is an inescapable, universal property of human language?

Or is this hierarchical progression simply an artifact of gradient descent discovering that local features are easier to optimize before global dependencies?

On the Clever Hans Effect: Attention, Explanation, and True Understanding

The lecture slides cite prominent papers debating interpretability: 'Attention is not Explanation' versus 'Attention is not not Explanation', as well as NLP's 'Clever Hans Moment'.

When we visualize an attention head connecting a pronoun to its referent with high weights (e.g., 94% accuracy on determiner relations), it is tempting to claim the model 'understands' syntactic dependencies.

But if an attention weight is merely an intermediate scaling factor in a non-linear network with residual streams and feed-forward projections, can we ever treat attention heatmaps as faithful explanations of why a model made a decision?

How can we distinguish whether BERT is genuinely comprehending linguistic meaning or merely exploiting dataset artifacts like a trained circus horse tapping its hoof?

References on Transformers & BERT

- <https://nlp.seas.harvard.edu/2018/04/03/attention.html>
- A. Vaswani et al., “Attention Is All You Need”, (2017). This title was so catchy that no less than 50 follow-up papers have included “all you need” in their titles!
- J. Howard and S. Ruder, “Universal Language Model Fine-Tuning for Text Classification”, (2018).
- A. Radford et al., “Improving Language Understanding by Generative Pre-Training”, (2018).
- J. Devlin et al., “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding”, (2018).
- I. Sutskever, O. Vinyals, and Q.V. Le, “Sequence to Sequence Learning with Neural Networks”, (2014).
- D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate”, (2014).
- A. Radford, R. Jozefowicz, and I. Sutskever, “Learning to Generate Reviews and Discovering Sentiment”, (2017).
- Y. Zhu et al., “Aligning Books and Movies: Towards Story-Like Visual Explanations by Watching Movies and Reading Books”, (2015).

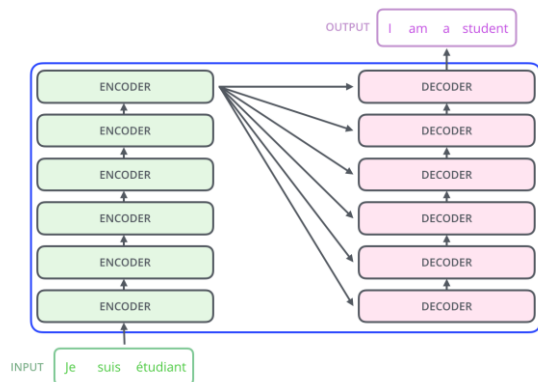
>>> Should you decide to read only 2 papers in this course, then make sure it will be these two!



BERT: Attention, Explanation & Understanding

- [1]: Attention is not Explanation, Sarthak Jain, Byron C. Wallace. <https://arxiv.org/abs/1902.10186>.
- [2]: Attention is not not Explanation. Sarah Wiegrefe and Yuval Pinter.
<https://arxiv.org/abs/1902.10186>.
- [3]: Is Attention Interpretable? Sofia Serrano and Noah A. Smith. <https://arxiv.org/abs/1906.03731>.
- [4]: Timothy Niven and Hung-Yu Kao. 2019. Probing neural network comprehension of natural language arguments. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pages 4658–4664, Florence, Italy. Association for Computational Linguistics (on Clever Hans effect). <https://www.aclweb.org/anthology/P19-1459/>
- [5]: NLP’s Clever Hans Moment has Arrived. Benjamin Heinzerling. August 26, 2019.
<https://thegradient.pub/nlps-clever-hans-moment-has-arrived/>.

Good (Visual) Tutorials on Transformers & BERT



1) This is our input sentence*

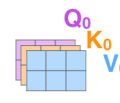
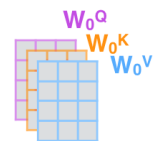
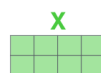
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

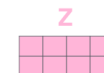
4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

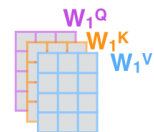
Thinking Machines



W^O



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



- <https://jalammar.github.io/illustrated-transformer/>
- <https://jalammar.github.io/a-visual-guide-to-using-bert-for-the-first-time/>